

Sistema de Gestão de Clínicas

Documentação de Engenharia de Software | Arquitetura de Microsserviços

Disciplina: Arquitetura e Padrões de Software

Instituição: Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET/RJ)

Professor: Marcelo Arêas

Curso: Engenharia de Computação / Engenharia de Software

1. Introdução e Definição do Problema

No atual cenário de clínicas multiespecialidades, a eficiência e a alta disponibilidade sistêmica são fundamentais. Para suportar a expansão do negócio, é necessário um software projetado para a resiliência operacional, assegurando que necessidades de manutenção ou falhas isoladas ocorram sem causar o desligamento de toda a plataforma. Caso tenham que optar por outras opções de software, podem sofrer com alguns problemas estruturais que acabam afetando os negócios, como:

- **Efeito Cascata em Falhas (Acoplamento Forte):** Se o módulo de faturação ou emissão de notas fiscais sofrer uma sobrecarga ou falha técnica, todo o sistema incluindo o agendamento de consultas críticas e o acesso ao registo de pacientes fica indisponível. Isso gera severos prejuízos financeiros e deteriora a experiência do paciente.
- **Dificuldade de Escalabilidade:** Especialidades médicas possuem dinâmicas e sazonalidades distintas. Por exemplo, a pediatria pode apresentar picos de agendamento em épocas de mudança climática. Num monólito, não é possível escalar horizontalmente apenas o fluxo de agendamentos; é necessário replicar toda a aplicação, desperdiçando recursos computacionais de base de dados e infraestrutura.
- **Lentidão na Evolução Tecnológica:** Alterações simples no fluxo de cancelamento de consultas exigem a homologação e a implementação de toda a plataforma, aumentando o risco de regressões em áreas críticas, como o histórico clínico do paciente.

2. A Solução: Arquitetura Baseada em Microsserviços

Para sanar estes problemas, foi projetado uma plataforma de gestão de clínicas estruturada sobre uma arquitetura de microsserviços totalmente independentes, isolados em contentores Docker e comunicando-se de forma leve via APIs REST utilizando o formato estruturado JSON.

O sistema divide-se em três microsserviços fundamentais:

Microsserviço	Responsabilidades e Funcionalidades	Atores Impactados
MS-Cadastros (Serviço de cadastros)	Gerencia as entidades do domínio de pessoas. Responsável pelo CRUD de dados pessoais, validação de documentos e histórico.	Pacientes, Médicos, Administradores
MS-Agendamentos: (Serviço de Agendamentos)	Core de agendamento, remarcação e cancelamento de consultas. Controla os estados da agenda e avalia a disponibilidade de horários.	Pacientes, Médicos e Administradores
MS-Faturamento: (Serviço de Faturamento/Cobrança)	Geração de faturas, emissão de cobranças de consultas realizadas e processamento financeiro guiado por regras.	Administradores

Fluxo Operacional dos Atores:

- Paciente: Realiza auto registro e edição de perfil (via MS-Cadastros), marca, remarca e solicita o cancelamento de consultas (via MS-Agendamentos).
- Administrador: Registra médicos e especialistas (via MS-Cadastros), avalia e aprova/reprova cancelamentos (via MS-Agendamentos) e emite faturação (via MS-Faturamento).

3. Preocupações Arquiteturais (Concerns)

O projeto foca em resolver e mitigar preocupações fundamentais de sistemas distribuídos de missão crítica:

1. **Tolerância a Falhas e Isolamento de Processos:** A queda total ou parcial de um microsserviço não deve impedir que as outras partes do sistema funcionem.
2. **Escalabilidade Elástica e Independente:** Capacidade de responder a picos de procura de forma isolada, otimizando o uso de infraestrutura.
3. **Independência de Implantação (Deploy):** Garantir que novas funcionalidades possam ir para produção sem a necessidade de paralisar o agendamento.

4. Pontos de Vista (Viewpoints) da Arquitetura

4.1. Ponto de Vista de Implantação

Cada microsserviço é encapsulado em um container Docker dedicado, garantindo isolamento total:

- Container 1 (**MS-Cadastrros**)
- Container 2 (**MS-Agendamentos**)
- Container 3 (**MS-Faturamento**)

4.2. Ponto de Vista de Desenvolvimento (API Design)

A comunicação entre os microsserviços ocorre estritamente via protocolo HTTP/S, utilizando o padrão REST com formato JSON.

Exemplo de criação de agendamento:

POST /consultas

```
{ "paciente_id": 99823, "medico_id": 1140, "data_hora":  
  "2026-07-15T14:30:00Z" }
```

Resposta (201 Created):

```
{ "id": 10542, "paciente_id": 99823, "medico_id": 1140,  
  "data_hora": "2026-07-15T14:30:00Z", "status": "PENDENTE" }
```

Exemplo de consulta de agendamento:

GET /consultas?skip=0&limit=2

Resposta (Response - 200 OK):

```
[ { "id": 10542, "paciente_id": 99823, "medico_id": 1140,  
  "data_hora": "2026-07-15T14:30:00Z", "status": "PENDENTE" },  
  
  { "id": 10543, "paciente_id": 88412, "medico_id": 2250,  
  "data_hora": "2026-07-15T15:30:00Z", "status": "CONFIRMADA" } ]
```

Resposta de Erro (Response - 404 Not Found):

```
{ "detail": "Consulta não encontrada" }
```

5. Perspectivas Arquiteturais

5.1. Perspectiva de Tolerância a Falhas

Para atingir o requisito de resiliência, são aplicados padrões de engenharia de software:

- **Cenário de Inoperabilidade do MS-Agendamentos:** Caso o serviço de agendamentos fique fora do ar, o restante do sistema funciona. Pacientes conseguem acessar e editar o seu perfil no MS-Cadastros, e os administradores continuam a emitir relatórios de faturação no MS-Faturamento ou a registrar médicos no MS-Cadastros.
- **Cenário de Inoperabilidade do MS-Faturamentos:** Se o serviço de faturação ficar offline, o fluxo central não é interrompido. Os pacientes continuam a realizar o fluxo de agendamento normalmente no MS-Agendamentos. As requisições de faturação são enfileiradas e processadas assim que o serviço for restabelecido.

5.2. Perspectiva de Escalabilidade

O MS-Agendamentos pode receber réplicas adicionais autoescaláveis nos dias de maior volume, dividindo a carga JSON por meio de um balanceador de carga, enquanto o MS-Cadastros mantém-se estável com uma infraestrutura mais leve.